

Minuta de Clase — Tema 04

ORM avanzado + puente a interfaz MVC

Materia: Laboratorio de Programación y Lenguajes · IF009 · UNTDF

Ciclo lectivo: 2026 · Semana 8

Duración total: 360 min = 6 horas (Clase Teórica 180 min + Clase Práctica 180 min)

Docente: Matías Gel

Estado: producción — generado por class-writer (Roberto) el 2026-04-29

Objetivos de la clase

1. **Analizar** el ciclo de vida de un `QuerySet` : lazy evaluation, caché interna, cuándo se evalúa.
 2. **Construir** consultas complejas usando Q objects, F expressions, `annotate()` y `aggregate()` .
 3. **Evaluar** el costo del problema N+1 e implementar `select_related` / `prefetch_related` .
 4. **Extender** managers personalizados del dominio Biblioteca al dominio BlogApp con `only()` y `defer()` .
 5. **Comprender** el ciclo request/response de Django y la responsabilidad de cada capa MVT.
 6. **Reconocer** `View` como clase base, `as_view()` , `dispatch()` , `get()` / `post()` .
 7. **Construir** templates con DTL completo: herencia, partials, filtros, `{% load static %}` , variables `forloop` .
-

CLASE TEÓRICA (180 min)

[F-00] Portada

Tiempo: 2 min

Qué decir:

- Mientras carga el proyector: "Hoy completamos el ORM y cruzamos a la interfaz — dos temas en una clase puente."
- Mostrar agenda en pizarra: T1 (QuerySet avanzado) → T2 (Q/F/aggregate) → T3 (CBV) → T4 (DTL).
- Contextualizar: "Todo lo que hagan hoy con los modelos lo van a ver aparecer en el navegador al final."

Conceptos clave: —

Preguntas anticipadas: ninguna en este punto

Transición: Pasar directo a F-01 sin demora.

[F-01] Puente pedagógico

Tiempo: 5 min

Qué decir:

- “Antes de arrancar, hagamos memoria de qué hicieron en la práctica anterior.”
- Repasar brevemente la columna izquierda: “Ya usaron `filter()`, `get()`, `order_by()`, `save()`, `delete()` — y crearon un Manager. Eso está incorporado.”
- “Hoy trabajamos con BlogApp — Post, Category, Comment. Mismo patrón ORM, dominio nuevo.”
- Señalar columna derecha: “Esto es genuinamente nuevo — lazy evaluation, Q objects, el N+1.”

Conceptos clave: continuidad pedagógica, dominio BlogApp

Preguntas anticipadas: “¿Usamos el mismo proyecto?” — No, hay un Codespace de BlogApp con el setup listo.

Transición: “Empecemos por entender mejor cómo funciona el QuerySet internamente.”

Bloque T1 — QuerySet API (45 min)

[F-02] ¿Qué es un QuerySet realmente?

Tiempo: 6 min

Qué decir:

- “Cuando hacen `Post.objects.filter(...)` — ¿qué obtienen? No los datos. Obtienen un *objeto* que *describe* la consulta.”
- “Es como escribir una pregunta en un papel — hasta que no se la mostrás a alguien, nadie la responde.”
- Dibujar en pizarra: `QuerySet` como una caja con una flecha punteada hacia la BD. La flecha se vuelve sólida solo cuando se itera.
- “Esto es lo que hace al ORM eficiente: no consulta si no hace falta.”

Conceptos clave: QuerySet diferido, lazy evaluation, caché interna

Preguntas anticipadas: “¿Entonces cuando hago `filter()` no tengo los datos?” → Exacto. Tenés el *plan* de obtenerlos.

Transición: “Veamos exactamente cuándo se ejecuta esa SQL.”

[F-03] Lazy evaluation en acción

Tiempo: 6 min

Qué decir:

- Mostrar el código paso a paso. "En la línea 1 — cero SQL. El ORM guardó la intención."
- "En el for loop — ahí sí. Django traduce el QuerySet a SQL y va a la BD."
- "Lo más importante: segunda iteración — Django tiene los datos en caché. Sin SQL extra."
- "¿Por qué importa esto? En el template, van a iterar sobre un QuerySet. Si ya fue evaluado en la vista, no se vuelve a consultar la BD."

Conceptos clave: evaluación diferida, caché de QuerySet, eficiencia

Preguntas anticipadas: "¿La caché se limpia?" → Se limpia cuando el QuerySet se modifica (filter, exclude, etc.). "¿Cuánto dura?" → Solo en la misma request/función.

Transición: "¿Cuándo exactamente se dispara esa evaluación? Hay 6 situaciones."

[F-04] Cuándo se evalúa un QuerySet

Tiempo: 5 min

Qué decir:

- Leer la tabla con ejemplos en voz alta. "Iteración — el más común. `list()` — explícito, útil en tests."
- "Slicing — si hacen `qs[0:5]`, ahí se ejecuta. Pero `qs[0:5]` sin consumir sigue siendo lazy."
- "El que más los va a sorprender: `if qs` — eso evalúa el QuerySet. Si solo quieren saber si hay resultados, usen `.exists()` que es más eficiente."

Conceptos clave: 6 puntos de evaluación, `.exists()` vs `if qs`

Preguntas anticipadas: "¿Y en el shell cuando tipeo el nombre?" → Sí, `repr()` lo evalúa — por eso ven los datos al tipear.

Transición: "Ahora, chaining — encadenar métodos."

[F-05] Chaining

Tiempo: 5 min

Qué decir:

- "Pueden encadenar `.filter()`, `.exclude()`, `.order_by()`, `.values()` — cada uno devuelve un nuevo QuerySet."
- Señalar el backslash: "El `\` es solo para continuar la línea en Python — no tiene significado para Django."
- "¿Cuándo se ejecuta la SQL aquí?" → Señalar `[:10]`. "El slicing es el disparo. Antes de eso, cero SQL."
- "Esto los deja construir la query paso a paso en la vista según las condiciones."

Conceptos clave: inmutabilidad del QuerySet, encadenamiento, momento de evaluación

Preguntas anticipadas: "¿El orden de los métodos importa?" → Para `filter` / `exclude` no. Para `order_by` el último gana. Para `values` / `only` debe ser antes del slicing.

Transición: "Vimos que podemos encadenar. Ahora los métodos nuevos que amplían el repertorio."

[F-06] Métodos nuevos

Tiempo: 5 min

Qué decir:

- “Esta tabla es el ‘qué más puedo hacer’. Léanla y anoten las que les llaman la atención.”
- Destacar: “ `exists()` — no trae objetos, solo verifica. Mucho más rápido que `if qs:` cuando solo importa si hay resultados.”
- “ `count()` — un `SELECT COUNT(*)` directo. No trae objetos.”
- “ `values()` y `values_list()` — cuando no necesitan el objeto completo, solo algunos campos.”
- “ `only()` y `defer()` — para modelos con campos grandes. `defer('body')` no trae el body del post a Python.”

Conceptos clave: eficiencia selectiva, `exists()`, `only()`, `defer()`

Preguntas anticipadas: “¿ `first()` lanza excepción si no hay?” → No, devuelve `None`. `get()` sí lanza `DoesNotExist`.

Transición: “Veamos `get_or_create` — muy común en código real.”

[F-07] `get_or_create` y `update_or_create`

Tiempo: 3 min

Qué decir:

- “Estos dos métodos son los favoritos en código de fixtures, scripts de carga, comandos de management.”
- “Devuelven una tupla: `(objeto, fue_creado)`. La variable `created` es un bool — `True` si acabó de crearse.”
- “Son atómicos a nivel Django — hacen primero el `SELECT`, luego el `INSERT` si no encuentra. En bases de datos con `UNIQUE` constraint hay que tener cuidado con race conditions en producción, pero para este nivel está bien.”

Conceptos clave: `(obj, created)`, atomicidad, `defaults` dict

Preguntas anticipadas: “¿Puedo usarlo con campos que no son únicos?” → Técnicamente sí, pero podés crear duplicados. Úsalo con campos que identifican unívocamente el objeto.

Transición: “Operaciones de escritura — `update()` y `bulk_create()`.”

[F-08] Escritura masiva

Tiempo: 5 min

Qué decir:

- “Acá hay una diferencia importante: `update()` no llama a `.save()` de cada instancia. Hace un `UPDATE ... WHERE` directo.”
- “¿Qué significa eso? Los signals de Django (`post_save`) no se disparan. Las validaciones del modelo tampoco. Son más rápidos pero ‘más crudos’.”
- “Para el nivel de este curso, `update()` y `bulk_create()` son perfectamente apropiados.”
- Mostrar el `delete()` : “El valor de retorno les dice exactamente qué borraron — útil para logging.”

Conceptos clave: `update()` sin `.save()`, `bulk_create()`, signals no se disparan

Preguntas anticipadas: “¿Y si necesito la validación?” → Iterar con `.save()` individual. “¿`bulk_create` tiene límite?” → En SQLite sí por el número de parámetros, en PostgreSQL prácticamente no.

Transición: “Técnicas para traer solo lo que necesitamos — `values()` y `only()`.”

[F-09 — F-10] `values()`, `values_list()`, `only()`, `defer()`

Tiempo: 5 min

Qué decir:

- F-09: “Si solo necesitan títulos para un dropdown de un formulario, ¿para qué traer todo el Post? `values_list()` les da solo los IDs y nombres.”
- F-10: “`only()` instancia el modelo pero sin todos los campos — el body del post puede tener miles de palabras. Si solo muestran el título en el listado, no lo traigan.”
- “`defer()` es el complemento: todo menos los campos pesados.”
- “Regla práctica: en vistas de listado, siempre pensá `only()`. En vistas de detalle, traer todo es razonable.”

Conceptos clave: eficiencia en transferencia de datos, N campos necesarios

Preguntas anticipadas: “¿Si accedo a un campo con `only()` que no pedí, se rompe?” → No se rompe, hace un SQL extra por el campo faltante. Por eso conviene pensar bien qué pedir.

Transición: “Cerramos T1 con el Manager personalizado — lo que hicieron en Biblioteca, ahora en BlogApp.”

[F-11] Manager personalizado en BlogApp

Tiempo: 4 min

Qué decir:

- “¿Recuerdan `LibroManager` con `disponibles`? Lo que vamos a hacer es idéntico en estructura.”
- “La diferencia es que ahora encapsulamos también la eficiencia: `select_related()` y `only()` dentro del Manager. Los que usen `Post.published.recientes()` automáticamente reciben los datos optimizados.”
- Señalar `objects = models.Manager()`: “Siempre declarar el manager por defecto cuando agregan un custom. Si no, desaparece y `Post.objects.all()` deja de funcionar.”

Conceptos clave: reutilización del patrón Manager, encapsular performance

Preguntas anticipadas: "¿Puedo tener varios Managers?" → Sí. `Post.objects` (todos) + `Post.published` (publicados).

Transición: Mostrar F-12 como síntesis y pasar a T2.

[F-12] Resumen §T1

Tiempo: 1 min **Qué decir:**

- Leer los 5 bullets en voz alta como repaso rápido.
- "¿Alguna pregunta antes de entrar a Q objects?" — responder solo si es muy puntual, pasar rápido.

Bloque T2 — Consultas dinámicas y performance (40 min)

[F-13] Pregunta socrática — el problema de filter()

Tiempo: 3 min

Qué decir:

- "Tengan esta situación: el usuario puede filtrar posts por categoría, por término de búsqueda, o quiere ver sus propios borradores además de los publicados."
- "Con `filter()` solo puedo `AND`. 'Publicado Y de esta categoría'. ¿Cómo hago 'publicado O del usuario actual'?"
- Esperar respuestas — algunos van a proponer dos QuerySets y unir con `|` en Python. "Casi — pero eso trae dos queries. ¿Podemos hacer una sola?"

Conceptos clave: limitación de `filter()`, necesidad de OR lógico

Preguntas anticipadas: la mayoría no sabe todavía — eso es el punto.

Transición: "La respuesta es Q objects."

[F-14 — F-15] Q objects

Tiempo: 8 min

Qué decir:

- F-14: "Q es un objeto que encapsula una condición. `|` es OR, `&` es AND, `~` es NOT."
- "Lo importante es que son combinables — podés guardar un Q en una variable y combinarlo con otro."
- F-15: "Este es el caso de uso más poderoso: construcción dinámica. El filtro empieza vacío `Q()` y vamos agregando condiciones según lo que el usuario pidió."
- "Una sola llamada a la BD al final — eficiente y legible."

Conceptos clave: Q como objeto de condición, operadores bitwise, construcción dinámica

Preguntas anticipadas: "¿Se pueden mezclar Q y filter() normal?" → Sí: `filter(Q(...), published=True)` funciona. Lo de filter() son ANDs implícitos.

Transición: "F expressions — operar sobre columnas de la BD sin traer datos."

[F-16] F expressions

Tiempo: 6 min

Qué decir:

- "Escenario real: tenemos un contador de vistas en cada Post. Cada vez que alguien lee un post, incrementamos `views`."
- "La forma naif: `post = Post.objects.get(pk=pk); post.views += 1; post.save()`. Dos queries, y hay un race condition — si dos usuarios leen el mismo post a la vez, ambos hacen `views += 1` sobre el mismo valor leído."
- "F expression: `Post.objects.filter(pk=pk).update(views=F('views') + 1)` — el incremento pasa en el SQL. Atómico. Sin race condition."

Conceptos clave: atomicidad, race conditions, F para comparar campos

Preguntas anticipadas: "¿F funciona con strings?" → Para concatenación hay `Concat` en `django.db.models.functions`.

Transición: "Aggregations — estadísticas."

[F-17 — F-18 — F-19] aggregate() y annotate()

Tiempo: 10 min

Qué decir:

- F-17: "aggregate() — una sola fila de resultado. Es el `SELECT COUNT(*), AVG(...)` del SQL. Devuelve un diccionario."
- F-18: "annotate() — agrega un campo calculado a *cada* objeto del QuerySet. La SQL usa `GROUP BY`. Cada Category ahora tiene `.post_count`."
- F-19: "La confusión más común: ¿cuándo uso cada uno? Si quiero 'cuántos posts tiene el blog' → `aggregate()`. Si quiero 'cuántos posts tiene cada categoría' → `annotate()`."
- "aggregate = estadística global. annotate = enriquecer cada objeto."

Conceptos clave: `aggregate()` devuelve dict, `annotate()` devuelve QuerySet enriquecido

Preguntas anticipadas: "¿Puedo usar annotate y después filter?" → Sí:

`Category.objects.annotate(n=Count('post')).filter(n__gt=5)`.

Transición: "Ahora el bug de performance más común en Django."

[F-20] El problema N+1

Tiempo: 5 min

Qué decir:

- “Este es el bug más común en aplicaciones Django en producción. Y el más silencioso — Django no avisa.”
- Mostrar el código: “1 query para traer todos los posts. Luego, en el loop, cada `post.author.username` dispara una query nueva porque Django tiene que ir a buscar el User.”
- “Con 10 posts: 11 queries. Con 1000 posts: 1001 queries. El servidor se va a piso.”
- “¿Por qué Django no hace el JOIN automáticamente? Porque así puede ser lazy — vos decidís cuándo optimizar.”

Conceptos clave: N+1 queries, ForeignKey lazy loading, diagnóstico

Preguntas anticipadas: “¿Pasa con todos los accesos a FK?” → Sí, con FK, O2O, y también con M2M (peor).

Transición: “La solución: `select_related` y `prefetch_related`.”

[F-21 — F-22] Solución N+1 + diagnóstico

Tiempo: 7 min

Qué decir:

- F-21: “`select_related` : Django hace un JOIN SQL. Todo en 1 query. Para FK y OneToOne.”
- “`prefetch_related` : Django hace 2 queries — una para posts, una para todos los comentarios de esos posts con `IN` . Para M2M y reverse FK.”
- “La combinación: `select_related('author').prefetch_related('categories')` — el QuerySet clásico de una vista de listado.”
- F-22: “Esto es lo que van a hacer en la práctica — `settings.DEBUG = True` y medir `len(connection.queries)` antes y después.”

Conceptos clave: JOIN vs queries separadas con IN, `select_related` para FK, `prefetch_related` para M2M

Preguntas anticipadas: “¿Cuándo usar `prefetch_related` para FK?” → Solo si el `select_related` no es posible (ej: GFK). Para FK directa, siempre `select_related` .

Transición: “Ejercicio de pizarra — detectar el N+1.”

[F-23] Evaluación formativa §T2

Tiempo: 5 min

Qué decir:

- “Vean este código — ¿cuántas queries se ejecutan si hay 50 posts con 3 comentarios cada uno?”
- Esperar respuestas: 1 (posts) + 50 (author por post) + 503 (*comments*) + 503 (comment.user) = 1 + 50 + 150 + 150 = 351.
- “¿Cómo lo corrigen? Una sola query.” →
`Post.objects.select_related('author').prefetch_related('comments__user')`.
- Escribir la solución en la pizarra.

Conceptos clave: calcular el costo de N+1 anidado, `prefetch_related` con doble underscore

Break: 5 min después de esta sección.

Bloque T3 — Puente MVC (25 min)

[F-24 — F-25] Ciclo request/response y responsabilidades

Tiempo: 8 min

Qué decir:

- “Hasta ahora trabajamos con la capa de datos — Models. Hoy cruzamos al otro lado.”
- Dibujar el diagrama en pizarra desde cero: Browser → [urls.py](#) → View → (Model + Template) → Response.
- “Esto se llama MVT en Django. No es exactamente MVC — el Template es el View del MVC clásico, y la View de Django es el Controller.”
- F-25: “La tabla de responsabilidades. Regla de oro: el Template no tiene lógica de negocio. El Model no conoce la request. La View orquesta sin dominio.”

Conceptos clave: MVT vs MVC, separación de responsabilidades

Preguntas anticipadas: “¿Django admin es una View?” → Sí, son vistas con CBV muy complejas.

Transición: “Veamos la View más simple — View base.”

[F-26] View como clase base

Tiempo: 7 min

Qué decir:

- “Esta es la CBV más simple posible. Sin magia — `get()` se llama cuando llega un GET.”
- Señalar `template_name`: “Es un atributo de clase — lo pueden sobrescribir en subclasses.”
- “El método `get()` recibe `request`. Consulta los modelos (ya saben cómo), pasa el contexto al template, devuelve el Response.”
- “Es OOP aplicado a HTTP: cada método HTTP es un método de la clase.”

Conceptos clave: `get()` para GET, `render()`, contexto como dict

Preguntas anticipadas: "¿Puedo poner lógica de negocio en la View?" → Lógica de presentación sí. Lógica de dominio no — va en el modelo o en un service.

Transición: "¿Cómo conectamos la clase con las URLs?"

[F-27] `as_view()` y `dispatch()`

Tiempo: 5 min

Qué decir:

- "Django espera un callable en `urls.py`. Una clase no es callable. `as_view()` crea el callable."
- "Cuando llega el request, `dispatch()` lee `request.method`, lo convierte a minúsculas, y llama el método correspondiente: GET → `get()`, POST → `post()`."
- "Si el método no existe en la View — `dispatch()` devuelve `405 Method Not Allowed`."
- "Esto es lo que van a ver en la evaluación formativa."

Conceptos clave: `as_view()` como adaptador, `dispatch()` como enrutador HTTP

Preguntas anticipadas: "¿ `as_view()` crea una instancia nueva por request?" → Sí, una por request. Thread-safe.

Transición: "PostDetailView — con `get_object_or_404`."

[F-28] PostDetailView

Tiempo: 3 min

Qué decir:

- "Dos diferencias respecto a PostListView: recibe `pk` en la URL y usa `get_object_or_404`."
- "`get_object_or_404` — si el Post no existe o no está publicado, devuelve 404 automáticamente. Sin try/except manual."
- "Ya incluye el `select_related` y `prefetch_related` — nunca exponemos N+1 en producción."

Conceptos clave: `get_object_or_404`, 404 automático, parámetros en URL

Preguntas anticipadas: —

Transición: F-29 rápido y luego F-30.

[F-29] Por qué View base y no genérica

Tiempo: 2 min

Qué decir:

- “Pregunta frecuente: ‘profe, vi que existe ListView en la docs’. Sí existe. En Semana 9 lo usan.”
 - “Hoy usamos View base porque cuando lleguen a ListView van a entender exactamente qué automatiza. Si arrancamos con ListView, sería magia negra.”
-

[F-30] Evaluación formativa §T3

Tiempo: 2 min

Qué decir:

- “Pregunta directa: ¿qué devuelve Django si llega un PUT a una View que solo tiene `def get()` ?”
 - Esperar respuesta: `405 Method Not Allowed` . “Exacto — `dispatch()` busca `put()` , no lo encuentra, devuelve 405.”
-

Bloque T4 — DTL completo (45 min)

[F-31] Los 4 constructos de DTL

Tiempo: 3 min

Qué decir:

- “Todo el template language de Django se reduce a 4 elementos. Si saben estos 4, saben DTL.”
- Señalar cada uno mientras lo leen: variable, filtro, tag, comentario.
- “Los más ricos son los tags — hay tags para bucles, condicionales, herencia, carga de librerías.”

Conceptos clave: 4 elementos de DTL

Transición: “Empecemos por las variables.”

[F-32] Variables y notación de punto

Tiempo: 4 min

Qué decir:

- “La vista pasa un dict Python al template — eso es el contexto.”
- “El punto resuelve en este orden: primero atributo, luego clave de dict, luego índice de lista, luego método callable.”
- “Práctica: `{{ post.author.username }}` — Django resuelve `post` , luego `.author` (FK, lazy), luego `.username` .”
- “Los atributos privados con `_` no son accesibles por seguridad.”

Conceptos clave: contexto como dict, resolución de punto, privacidad

Preguntas anticipadas: "¿Puedo llamar métodos con argumentos?" → No, solo callables sin argumentos. Si necesitan pasar argumentos, calculen en la vista.

Transición: "Filtros — transformar datos al mostrar."

[F-33 — F-34] Filtros y auto-escape

Tiempo: 6 min

Qué decir:

- F-33: Leer la tabla de filtros. "Estos son los que van a usar el 90% del tiempo."
- Destacar `date` : "El formato `d/m/Y` — `d` es día, `m` es mes, `Y` es año con 4 dígitos."
- `truncatewords` : "Perfecto para cards de listado — no muestran todo el post."
- F-34: "Esto es importante para la seguridad. Django escapa HTML por defecto — si un usuario pone `<script>` en un campo, se escapa y no se ejecuta."
- "Si confían en el origen (ustedes escribieron el contenido), pueden usar `|safe` . Nunca con input de usuarios."

Conceptos clave: filtros más útiles, auto-escape, XSS

Preguntas anticipadas: "¿Cómo encadeno filtros?" → `{{ texto|lower|truncatewords:30 }}` — izquierda a derecha.

Transición: " `{% for %}` — el tag más usado."

[F-35 — F-36] for y forloop

Tiempo: 6 min

Qué decir:

- F-35: "El `{% for %}` funciona como el for de Python. El `{% empty %}` es lo nuevo — se muestra cuando la lista está vacía."
- "Sin `{% empty %}` tendrían que hacer un `{% if posts %}...{% else %}...{% endif %}` envolviendo el for. `{% empty %}` es más elegante."
- F-36: "Las variables de `forloop` son gratis — Django las inyecta en cada iteración."
- Destacar `forloop.first` y `forloop.last` : "Perfecto para destacar el primero o agregar separadores entre elementos."
- `forloop.counter` : "Numeración automática — no tienen que mantener un contador manual."

Conceptos clave: `{% empty %}` , variables `forloop` , numeración automática

Preguntas anticipadas: "¿ `forloop` funciona en loops anidados?" → Sí, `forloop.parentloop` accede al loop de afuera.

Transición: " `{% if %}` con todos los operadores."

[F-37 — F-38] if y trampa de precedencia

Tiempo: 4 min

Qué decir:

- F-37: "Los operadores son los mismos de Python — `==`, `!=`, `and`, `or`, `not`, `in`."
- "Lo nuevo: `{% elif %}`. Y que se puede usar `in` con listas y strings."
- F-38: "La trampa más importante. En Python, `and` tiene precedencia sobre `or`. En DTL, se evalúa de izquierda a derecha."
- "`a or b and c` en Python = `a or (b and c)`. En DTL = `(a or b) and c`. ¡Diferente resultado!"
- "Solución: cuando necesiten lógica compleja, anidar `{% if %}`."

Conceptos clave: precedencia izquierda-a-derecha en DTL vs Python

Preguntas anticipadas: "¿No hay paréntesis en DTL?" → No. Por eso se anida.

Transición: "`{% with %}` — alias de variables."

[F-39] with

Tiempo: 4 min

Qué decir:

- "¿Por qué `{% with %}`? Porque cada `{{ post.author }}` en el template puede ser un SQL si no usaron `select_related` en la vista."
- "Con `{% with %}`, hacen el lookup una sola vez y lo guardan en `author`. Dentro del bloque, siempre es la misma referencia."
- "También útil para acortar expresiones largas: `{% with total=business.employees.count %}`."

Conceptos clave: alias, evitar lookups repetidos, scope del bloque

Transición: "Librerías de tags: `{% load %}`."

[F-40 — F-41] load, static, url

Tiempo: 5 min

Qué decir:

- F-40: "Antes de usar tags de una librería, hay que cargarla. `{% load static %}` habilita `{% static %}`."
- "Lo importante: `{% load %}` no se hereda. Si `base.html` hace `{% load static %}`, los hijos deben hacerlo también si usan `{% static %}`."
- F-41: "`{% url %}` resuelve la URL por el nombre — no hardcodear `/blog/post/1/`."
- "Si cambian la URL en `urls.py`, todos los templates siguen funcionando. Si hardcodean, deben actualizar todos los templates."

Conceptos clave: `{% load %}` no se hereda, `{% url %}` vs hardcoded URL

Preguntas anticipadas: "¿Cómo nombro las URLs?" → `app_name = 'blog'` en `urls.py` + `name='post-list'` → `'blog:post-list'`.

Transición: "Comentarios — breve."

[F-42] comment

Tiempo: 1 min

Qué decir:

- "Rápido: `{# #}` para una línea, `{% comment %}` para un bloque."
 - "La diferencia con los comentarios HTML `<!-- -->`: estos sí llegan al navegador y se ven en 'Ver código fuente'. Los de DTL no."
 - "Truco útil: usar `{% comment %}` para desactivar secciones temporalmente sin borrar código."
-

[F-43 — F-44 — F-45] Herencia de templates

Tiempo: 7 min

Qué decir:

- F-43: "El problema: sin herencia, cada template tiene el mismo `<head>`, navbar, footer. Si cambian el logo, tocan 20 archivos."
- "La solución: un `base.html` con `{% block %}` para las partes variables. Los hijos heredan la estructura y reemplazan solo los bloques."
- F-44: Mostrar el código. "Lo más importante: `{% extends %}` debe ser la **primera línea** del template hijo. Si hay un comentario antes, falla."
- "`{% block content %}{% endblock %}` — el contenido por defecto del bloque. Si el hijo no lo sobreescribe, se usa este."
- F-45: "`{{ block.super }}` — el patrón más poderoso. Incluye el contenido del bloque padre y agrega el propio. Sin esto, sobreescribís y perdés lo del padre."

Conceptos clave: `{% extends %}` primero, `{% block %}`, `{{ block.super }}`

Preguntas anticipadas: "¿Puedo heredar de un template que ya hereda de otro?" → Sí, herencia multinivel es válida.

Transición: "`{% include %}` — partials."

[F-46] include

Tiempo: 2 min

Qué decir:

- “Mientras `{% extends %}` hereda el esqueleto, `{% include %}` inserta un fragmento — como un componente.”
- “La tarjeta de post es el ejemplo clásico. Si la pantalla de búsqueda, la homepage y el blog listing muestran la misma tarjeta, un solo archivo `post_card.html` es suficiente.”
- “El `with post=post` pasa la variable al partial. Dentro del partial, `post` existe.”

Conceptos clave: partial reutilizable, `with` en include

Transición: Mini-ejercicio.

[F-47] Mini-ejercicio §T4

Tiempo: 5 min

Qué decir:

- “5 minutos — individual, sin mirar las slides.”
 - “Escriban en papel o en el editor: un template que cumpla los 5 puntos.”
 - Después de 3 min: pedir a un estudiante que comparta. Revisar juntos.
-

[F-48] Cierre de clase teórica

Tiempo: 5 min

Qué decir:

- Leer los tres bullets del resumen.
 - “En la práctica van a conectar todo esto: van a escribir los mismos QuerySets que vieron en T1/T2 pero desde una vista, y van a ver el resultado en el navegador con los templates que construyan.”
 - “Anuncio: TP-5 cubre todo lo de hoy — ORM avanzado + primera vista + DTL. La consigna se va a publicar esta semana.”
 - “Semana 9: Parcial 1 al inicio, luego refactorizamos las vistas a `ListView` / `DetailView` y agregamos `ModelForm`.”
 - Break de 10 min antes de la práctica.
-

CLASE PRÁCTICA (180 min)

[F-49] Portada — Práctica

Tiempo: 2 min

Qué decir:

- “Hoy conectan todo lo que vieron en la teórica. El objetivo concreto: al final de la clase, van a tener el blog corriendo en el navegador.”
 - “Dos partes: §P1 en el shell (60 min) y §P2 construyendo vistas y templates (105 min).”
-

[F-50] Setup

Tiempo: 8 min

Qué decir:

- “Primero: abrir el Codespace de BlogApp. Si no lo tienen configurado, avisar ahora.”
- Esperar que todos estén listos. “Ejecutar `python manage.py migrate` para asegurarse.”
- “Verificar datos en el shell: `Post.objects.count()` — si es 0, ejecutar el script de fixtures que está en `scripts/seed.py`.”
- Esperar confirmación de todos antes de continuar.

Conceptos clave: entorno listo, datos de prueba disponibles

Preguntas anticipadas: “No me abre el Codespace” → Verificar que el Codespace está en la rama correcta y que la imagen está construida.

§P1 — Shell avanzado (60 min)

[F-51] Ejercicio 1

Tiempo: 10 min

Qué decir:

- “Ejercicio 1: reproduzcan este código en su shell.”
- Circular por el aula mientras trabajan. Revisar que estén usando `Category.objects.get()` y `filter().first()`.
- Después de 7 min: “¿Qué devuelve `filter().first()` si no hay ninguna categoría con ese slug?” → `None`.
- “¿Qué devuelve `get()` en ese caso?” → `DoesNotExist`. “¿Cuál es más seguro?” → Depende del contexto.

Conceptos clave: `get()` vs `filter().first()`, `exists()`, `count()`

Preguntas anticipadas: Ver arriba.

[F-52] Ejercicio 2

Tiempo: 10 min

Qué decir:

- "Q objects en acción. El truco: `print(qs.query)` — pueden ver el SQL que Django generó."
- Circular. Verificar que el OR está funcionando correctamente.
- "¿Alguien vio el SQL? ¿Qué tiene el WHERE clause?" → Debe tener `OR`.

Conceptos clave: SQL generado, `qs.query` para debugging

Preguntas anticipadas: "El `Q()` vacío — ¿qué hace?" → Es el elemento neutro del AND, no filtra nada.

[F-53] Ejercicio 3

Tiempo: 10 min

Qué decir:

- "Aggregations. `aggregate()` devuelve un dict — accedan a `total['total']`."
 - "El `annotate()` es el más importante: cada objeto de `categories` ahora tiene `.n_posts`. Django añadió un campo calculado."
 - Circular para verificar que el `for cat in categorias` esté funcionando y mostrando valores.
-

[F-54] Ejercicio 4 y 5

Tiempo: 15 min

Qué decir:

- "F expressions — después del `update()`, verifiquen que el `views` del Post aumentó."
- "El diagnóstico N+1 es el ejercicio más importante. Antes de correr el `select_related`, anoten cuántas queries salen. Después, cuántas."
- "Alguien que comparta los resultados — ¿cuántas sin optimizar, cuántas con?"

Conceptos clave: atomicidad de F, diagnóstico con `connection.queries`

Preguntas anticipadas: "El print muestra siempre 0" → Verificar que `settings.DEBUG = True` esté activo.

[F-55] Ejercicio 6

Tiempo: 10 min

Qué decir:

- “Agregar el `PublishedManager` al modelo `Post`. Ejecutar `Post.published.recientes(5)` y verificar que devuelve solo los publicados, ordenados.”
 - “Preguntar: ¿cuántas queries genera `recientes()` ? ¿Por qué solo 1?” → `select_related` en el Manager.
-

[F-56] Break + transición

Tiempo: 5 min

Qué decir:

- “Break de 5 min. En §P2 vamos a usar todas las consultas que acaban de escribir, pero llamadas desde una vista en lugar del shell.”
-

§P2 — Vista OOP + DTL (105 min)

[F-57] Estructura de archivos

Tiempo: 5 min

Qué decir:

- “Primero crear la estructura de carpetas. La carpeta `templates/blog/` dentro de la app es la convención de Django para `APP_DIRS=True`.”
 - “Si usan otra ruta, Django no la va a encontrar automáticamente.”
-

[F-58] PostListView

Tiempo: 10 min

Qué decir:

- “Copiar o escribir el código de `PostListView`. Notar que el QuerySet ya incluye `select_related` y `prefetch_related`.”
 - “¿Por qué incluirlos aquí? Porque el template va a acceder a `post.author.username` — si no los incluimos, N+1.”
 - Verificar que el `urls.py` está conectado antes de continuar.
-

[F-59] PostDetailView + URLs

Tiempo: 10 min

Qué decir:

- “Agregar `PostDetailView` . El `get_object_or_404` simplifica el try/except.”
 - “Conectar en `blog/urls.py` y verificar que `blogapp/urls.py` incluye `blog.urls` con `include()` .”
 - “Correr `python manage.py runserver` y navegar a `/blog/` . Van a ver un error de template — está bien, el template todavía no existe.”
-

[F-60] base.html

Tiempo: 10 min

Qué decir:

- “El esqueleto. Lo más importante: `{% load static %}` primero, y los tres `{% block %}` básicos: `title` , `content` , `extra_head` .”
 - “Verificar que `STATICFILES_DIRS` está en `settings.py` .”
 - “El `{% now "Y" %}` en el footer — demuestra que hay tags de utilidad además de los de control de flujo.”
-

[F-61] post_list.html y post_detail.html

Tiempo: 15 min

Qué decir:

- “Crear ambos templates. `{% extends %}` debe ser la primera línea — literalmente la primera, sin espacios antes.”
 - “Navegar a `/blog/` → deben ver el listado. Clic en un post → ver el detalle.”
 - “Verificar que el navbar aparece en ambas páginas — eso confirma que la herencia funciona.”
-

[F-62] with en la tarjeta

Tiempo: 8 min

Qué decir:

- “Editar `post_list.html` para envolver el contenido del for en `{% with author=post.author %}` .”
 - “Usar `author.get_full_name|default:author.username` — muestra el nombre completo si existe, si no el username.”
-

[F-63] forloop

Tiempo: 8 min

Qué decir:

- “Agregar las variables de `forloop` . `forloop.first` para la clase `featured` , `forloop.counter` para el número, `forloop.last` para el mensaje final.”
 - “Refrescar el navegador — ¿el primer post tiene clase `featured` ? Verificar con las DevTools.”
-

[F-64] Partial `post_card.html`

Tiempo: 10 min

Qué decir:

- “Crear la carpeta `partials/` dentro de `templates/blog/` .”
 - “Mover el HTML de la tarjeta al partial. Reemplazar el contenido del `for` en `post_list.html` por el `{% include %}` .”
 - “Refrescar — el resultado debe ser idéntico. La diferencia es interna: ahora la tarjeta es reutilizable.”
-

[F-65] CSS con `{% static %}`

Tiempo: 10 min

Qué decir:

- “Crear `static/blog/css/styles.css` con el CSS mínimo.”
 - “Verificar `settings.py` — `STATICFILES_DIRS = [BASE_DIR / 'static']` .”
 - “En `base.html` ya pusimos `{% static 'blog/css/styles.css' %}` . Si todo está bien, al refrescar el CSS se aplica.”
-

[F-66] `{% comment %}` y verificación

Tiempo: 5 min

Qué decir:

- “Agregar comentarios DTL en cualquier template.”
 - “Ctrl+U en el navegador — ver el HTML fuente. Buscar los comentarios: **no deben aparecer**.”
 - “Esto es distinto a los comentarios HTML `<!-- -->` que sí aparecen.”
-

[F-67] Ticket de salida

Tiempo: 5 min

Qué decir:

- “5 minutos. Individual. En papel o en el chat.”
 - “3 tags DTL usados hoy — nombre y una oración explicando qué hace cada uno.”
 - Recolectar respuestas. Leer algunas en voz alta como cierre.
-

[F-68] Cierre — práctica

Tiempo: 5 min

Qué decir:

- “Repaso: ¿qué construyeron hoy?”
 - Leer los bullets del resumen.
 - “TP-5 va a cubrir exactamente esto — van a tener una consigna que extiende el BlogApp.”
 - “Semana 9: Parcial 1 al inicio (15 min), luego `ListView` / `DetailView` y `ModelForm`.”
 - “¿Preguntas finales?”
-

Cierre general

Resumen de la clase:

1. ORM: lazy evaluation, Q/F, aggregate/annotate, N+1 → `select_related/prefetch_related`
2. CBV: `View` base, `as_view()`, `dispatch()`, `get_object_or_404`
3. DTL: variables, filtros, `{% for %}` + `forloop`, `{% if %}`, `{% with %}`, `{% static %}`, herencia, `partials`

Anuncio del TP: TP-5 — BlogApp: ORM avanzado + primera vista + DTL completo. Consigna esta semana.

Próxima clase (Semana 9): Parcial 1 → `ListView` / `DetailView` → `ModelForm` con PRG